

Algorithms and Data Structures

10.02.2025

Exercise 1 (4.0 points)

Given an array of 2^n elements, the following function partitions it into sub-arrays of half the size until all sub-arrays have a length equal to one cell.

```
void show (int v[], int l, int r) {
    int i, c;
    if (l >= r) {
        return;
    }
    c = (r+l)/2;
    show (v, l, c);
    show (v, c+1, r);
    return;
}
```

Write the recurrence equation of the recursive function, solve it through unfolding and substitutions, and express its asymptotic complexity.

Solution

Please see the teacher's overhead for a complete analysis of the topic.

Exercise 2 (1.5 points)

The following expression is given in in-fix notation. Using its representation as a binary tree, convert it into pre-fix (Polish) and post-fix (Reverse Polish) notation.

$(A * B - (C * (D + E))) - (((F + G) * (H - I)) / L)$

Report the expression in pre-fix notation and post-fix notation. Notice that this is an open question that will be manually evaluated.

Solution

```
### Expression evaluation
Original expression: ( A * B - ( C * ( D + E ) ) ) - ( ( ( F + G ) * ( H - I ) ) / L )
Infix notation      : A * B - C * D + E - F + G * H - I / L
Prefix notation     : - - * A B * C + D E / * + F G - H I L
Postfix notation    : A B * C D E + * - F G + H I - * L / -
```

Response

```
- - * A B * C + D E / * + F G - H I L
A B * C D E + * - F G + H I - * L / -
```

Exercise 3 (1.5 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=23$. Insertions occur character by character using open addressing with double hashing. Use functions $h_1(k) = k \% M$ and $h_2(k) = 1 + (k \% 97)$. Each character is identified by its index in the English alphabet (i.e., A=1, ..., Z=26). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

E U E U E U

Indicate in which elements the last three letters of the sequence are placed, i.e., U, E, and U, in this order. Please report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. The following is an example of the response format: 3 4 11

Solution

```
### hash-table
Character keys
Hash table size = 23
Number of keys  = 6
Double hashing.
Key:  E U E U E U
Key=E k=005 Final=05
Key=U k=021 Final=21
Key=E k=005 Coll.= 5 Final=11
Key=U k=021 Coll.=21 Final=20
Key=E k=005 Coll.= 5 Coll.=11 Final=17
Key=U k=021 Coll.=21 Coll.=20 Final=19
00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22
-  -  -  -  -  E  -  -  -  -  -  E  -  -  -  -  -  E  -  U  U  U  -
```

Response

20 17 19

Exercise 4 (1.5 points)

Given the following sequence of integers stored in an array, turn it in a heap, assuming to use an array as an underlying data structure. Assume that, in the end, the largest value is stored at the heap's root. Then, execute the first two steps of heapsort on the heap built at the previous step.

9 13 10 8 5 7 18 2 6

Report the final content of the entire array at the end of the above process. Please show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response. The following is an example of the response: 0 3 2 6 8 etc.

Solution

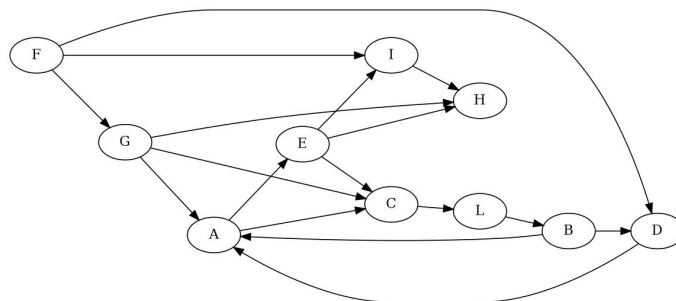
```
### Heap sort
-----
Initial array:  0  1  2  3  4  5  6  7  8
heap-build
-----
heapify        No swap
                9 13 10  8  5  7 18  2  6
heapify        [2<->6]
                9 13 18  8  5  7 10  2  6
heapify        No swap
                9 13 18  8  5  7 10  2  6
heapify        [0<->2] [2<->6]
                18 13 10  8  5  7  9  2  6
heap-sort
-----
heapify        [0<->1] [1<->3]
                13  8 10  6  5  7  9  2 18
heapify        [0<->2] [2<->6]
                10  8  9  6  5  7  2 13 18
heapify        [0<->2] [2<->5]
                9  8  7  6  5  2 10 13 18
heapify        [0<->1] [1<->3]
                8  6  7  2  5  9 10 13 18
heapify        [0<->2]
                7  6  5  2  8  9 10 13 18
heapify        [0<->1]
                6  2  5  7  8  9 10 13 18
heapify        no-swaps
                5  2  6  7  8  9 10 13 18
heapify        no-swaps
                2  5  6  7  8  9 10 13 18
```

Response

10 8 9 6 5 7 2 13 18

Exercise 5 (1.5 points)

Visit the following graph depth-first, starting at node A. Label nodes with discovery and end-processing times. Start with the discovery time set to 1 on A. When necessary, consider nodes and edges in alphabetic order.



Display the end-processing time of all vertices. Please indicate the end-processing time of all vertices sorted in alphabetic order (i.e., display the end-processing time for A B C D etc.). Report a sequence of integer values separated

by one single space. No other symbols must be included in the response. The following is an example of the response:
15 13 2 16 8 etc.

Solution

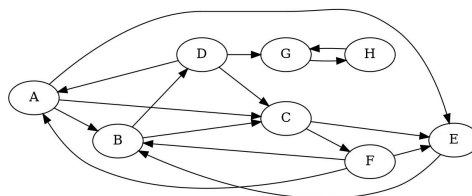
```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] L
Adjacency Matrix:
0 0 1 0 1 0 0 0 0 0
1 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0
0 0 1 0 0 0 0 1 1 0
0 0 0 1 0 0 1 0 1 0
1 0 1 0 0 0 0 1 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0
0 1 0 0 0 0 0 0 0 0
### Graph visit
BFS:
Node=[ 0] A Discovery_time= 0 Predecessor=[-1]
Node=[ 1] B Discovery_time= 3 Predecessor=[ 9]
Node=[ 2] C Discovery_time= 1 Predecessor=[ 0]
Node=[ 3] D Discovery_time= 4 Predecessor=[ 1]
Node=[ 4] E Discovery_time= 1 Predecessor=[ 0]
Node=[ 5] F Discovery_time= - Predecessor=[-1]
Node=[ 6] G Discovery_time= - Predecessor=[-1]
Node=[ 7] H Discovery_time= 2 Predecessor=[ 4]
Node=[ 8] I Discovery_time= 2 Predecessor=[ 4]
Node=[ 9] L Discovery_time= 2 Predecessor=[ 2]
DFS:
List of edges:
[ 0] A -> [ 2] C : (T) Tree
[ 2] C -> [ 9] L : (T) Tree
[ 9] L -> [ 1] B : (T) Tree
[ 1] B -> [ 0] A : (B) Back
[ 1] B -> [ 3] D : (T) Tree
[ 3] D -> [ 0] A : (B) Back
[ 0] A -> [ 4] E : (T) Tree
[ 4] E -> [ 2] C : (C) Cross
[ 4] E -> [ 7] H : (T) Tree
[ 4] E -> [ 8] I : (T) Tree
[ 8] I -> [ 7] H : (C) Cross
[ 5] F -> [ 3] D : (C) Cross
[ 5] F -> [ 6] G : (T) Tree
[ 6] G -> [ 0] A : (C) Cross
[ 6] G -> [ 2] C : (C) Cross
[ 6] G -> [ 7] H : (C) Cross
[ 5] F -> [ 8] I : (C) Cross
List of vertices:
[ 0] A: dist_time= 1 endp_time=16 pred=[-1] -
[ 1] B: dist_time= 4 endp_time= 7 pred=[ 9] L
[ 2] C: dist_time= 2 endp_time= 9 pred=[ 0] A
[ 3] D: dist_time= 5 endp_time= 6 pred=[ 1] B
[ 4] E: dist_time=10 endp_time=15 pred=[ 0] A
[ 5] F: dist_time=17 endp_time=20 pred=[-1] -
[ 6] G: dist_time=18 endp_time=19 pred=[ 5] F
[ 7] H: dist_time=11 endp_time=12 pred=[ 4] E
[ 8] I: dist_time=13 endp_time=14 pred=[ 4] E
[ 9] L: dist_time= 3 endp_time= 8 pred=[ 2] C
Reachable nodes from vertex [ 0] A:
[ 0] A - [ 1] B - [ 2] C - [ 3] D - [ 4] E - [ 5] F - [ 6] G - [ 7] H - [ 8] I - [ 9] L -
UN-reachable nodes from vertex [ 0] A:
none
```

Response

16 7 9 6 15 20 19 12 14 8

Exercise 6 (1.5 points)

Given the following directed graph, represent the reverse graph and find all strongly connected components.



Compute all strongly connected components in the graph. Report them in alphabetic order, and within each component, indicates vertices in alphabetic order (i.e., ACZ and not CAZ or ZAC). Separate components with a single

space. No other symbols must be included in the response. The following is an example of the response format: ABC
EF XY etc.

Solution

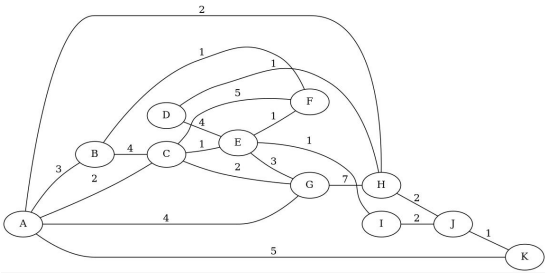
```
Directed matrix ... stated as such.
UN-weighted matrix ...stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H
Adjacency Matrix:
0 1 1 0 1 0 0 0
0 0 1 1 0 0 0 0
0 0 0 0 1 1 0 0
1 0 1 0 0 0 0 1
0 1 0 0 0 0 0 0
1 1 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 1 0
### Graph SCC
Visit on G^T:
[ 0] A: dist_time= 1 endp_time=12
[ 1] B: dist_time= 3 endp_time=10
[ 2] C: dist_time= 5 endp_time= 6
[ 3] D: dist_time= 2 endp_time=11
[ 4] E: dist_time= 4 endp_time= 9
[ 5] F: dist_time= 7 endp_time= 8
[ 6] G: dist_time=13 endp_time=16
[ 7] H: dist_time=14 endp_time=15
Visit on G:
[ 0] A: dist_time= 5 endp_time=16
[ 1] B: dist_time= 6 endp_time=15
[ 2] C: dist_time= 7 endp_time=12
[ 3] D: dist_time=13 endp_time=14
[ 4] E: dist_time= 8 endp_time= 9
[ 5] F: dist_time=10 endp_time=11
[ 6] G: dist_time= 1 endp_time= 4
[ 7] H: dist_time= 2 endp_time= 3
Number of SCCs: 2
SCC0: [ 6] G [ 7] H
SCC1: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F
```

Response

ABCDEF GH

Exercise 7 (1.5 points)

Given the following undirected and weighted graph, find a minimum spanning tree using Kruskal’s algorithm.



Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. The following is an example of the response format:

Solution

```
UN-directed matrix ... stated as such.
Weighted matrix ... stated as such.
Vertex list: [ 0] A [ 1] B [ 2] C [ 3] D [ 4] E [ 5] F [ 6] G [ 7] H [ 8] I [ 9] J [10] K
Adjacency Matrix:
0 3 2 0 0 0 4 2 0 0 5
3 0 4 0 0 1 0 0 0 0 0
2 4 0 0 1 5 2 0 0 0 0
0 0 0 0 4 0 0 1 0 0 0
0 0 1 4 0 1 3 0 1 0 0
0 1 5 0 1 0 0 0 0 0 0
4 0 2 0 3 0 0 7 0 0 0
2 0 0 1 0 0 7 0 0 2 0
0 0 0 0 1 0 0 0 0 2 0
0 0 0 0 0 0 2 2 0 1
5 0 0 0 0 0 0 0 0 1 0
### Graph MST (Kruskal and Prim)
Kruskal
Sorted array of edges: src dst weight:
```

	B	F	1
	C	E	1
	D	H	1
	E	F	1
	E	I	1
	J	K	1
	A	C	2
	A	H	2
	C	G	2
	H	J	2
	I	J	2
	A	B	3
	E	G	3
	A	G	4
	B	C	4
	D	E	4
	A	K	5
	C	F	5
	G	H	7

```

MST (list of edges):
Edge [ 1] B - [ 1] F ==> weight = 1
Edge [ 2] C - [ 2] E ==> weight = 1
Edge [ 3] D - [ 3] H ==> weight = 1
Edge [ 4] E - [ 4] F ==> weight = 1
Edge [ 4] E - [ 4] I ==> weight = 1
Edge [ 9] J - [ 9] K ==> weight = 1
Edge [ 0] A - [ 0] C ==> weight = 2
Edge [ 0] A - [ 0] H ==> weight = 2
Edge [ 2] C - [ 2] G ==> weight = 2
Edge [ 7] H - [ 7] J ==> weight = 2
Total tree weight      = 14

Prim
MST (list of edges):
Edge [ 0] A - [ 2] C ==> weight = 2
Edge [ 2] C - [ 4] E ==> weight = 1
Edge [ 4] E - [ 5] F ==> weight = 1
Edge [ 5] F - [ 1] B ==> weight = 1
Edge [ 4] E - [ 8] I ==> weight = 1
Edge [ 2] C - [ 6] G ==> weight = 2
Edge [ 0] A - [ 7] H ==> weight = 2
Edge [ 7] H - [ 3] D ==> weight = 1
Edge [ 8] I - [ 9] J ==> weight = 2
Edge [ 9] J - [10] K ==> weight = 1
Total tree weight      = 14

```

Response

14

Exercise 8 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```

#define L 100
void f (char *s1, char *s2, char *result) {
    int len1 = strlen(s1), len2 = strlen(s2);
    int i = 0, j = 0, k = 0;
    while (i < len1 && j < len2) {
        if (s1[i] < s2[j]) {
            result[k++] = s1[i++];
            result[k++] = s2[j++];
        } else {
            result[k++] = s2[j++];
            result[k++] = s1[i++];
        }
    }
    while (i < len1) {
        result[k++] = s1[i++];
    }
    while (j < len2) {
        result[k++] = s2[j++];
    }
    result[k] = '\0';
}

int main() {
    char s1[L] = "019339";
    char s2[L] = "394171";
    char result[2 * L];
    f (s1, s2, result);
    printf("%s", result);
    return 0;
}

```

Response

031949133719

Exercise 9 (2.0 points)

The following function deletes the node with the given key from a BST rooted at node root.

```
node_t *delete_r (node_t *root, char *key) {
    node_t *p;
    char *val;
    if (root == NULL)
        return (root);
    LINE 1 {
        root->l = delete_r (root->l, key);
        return (root);
    }
    if (strcmp (key, root->key) > 0) {
        root->r = delete_r (root->r, key);
        return (root);
    }
    p = root;
    if (root->r == NULL) {
        root = root->l;
        free (p);
        return (root);
    }
    if (root->l == NULL) {
        root = root->r;
        free (p);
        return (root);
    }
    root->l = f (&val, root->l);
    root->key = val;
    return (x); // LINE 2
}
node_t *f (char *val, node_t *root) {
    node_t tmp;
    if (root->r == NULL) {
        *val = root->key;
        tmp = x; // LINE 3
        free (root);
        return (tmp);
    }
    LINE 4
    return (root);
}
```

Indicate which one of the following statements is correct. Note that incorrect answers imply a penalty in the final score.

1. Even if the BST is not balanced, the function has a time complexity equal to $O(h)$ where h is the height of the tree.
2. Even if the BST is not balanced, the function has a time complexity equal to $O(\log(n))$ where n is the number of nodes stored in the tree.
3. Line 1 must include the test `if (strcmp (key, root->key) <= 0)`.
4. In Line 2, `x` must be substituted by `val`.
5. The function `f` finds the maximum value in the left subtree.
6. The function `f` finds the minimum value in the right subtree.
7. In Line 3, `x` must be equal to `root->l`.
8. In Line 3, `x` must be equal to `root->r`.
9. Line 4 must be equal to `root->r = f (val, root->r);`.
10. Line 4 must be equal to `root->l = f (val, root->l);`.

Response

1. TRUE.
2. FALSE.
3. FALSE.
4. FALSE.
5. TRUE.
6. FALSE.
7. TRUE.
8. FALSE.
9. TRUE.

10. FALSE.

Exercise 10 (2.0 points)

Analyze the following program and indicate the exact output it generates. Please report the exact program output with no extra symbols.

```
void plr (char *str, int left, int right);
void prl (char *str, int left, int right);
void pm (char *str, int left, int right);
int main() {
    char str[] = "HELLOWORLD";
    int length = strlen(str);
    plr (str, 0, length - 1);
    return 0;
}
void plr (char *str, int left, int right) {
    if (left > right) return;
    printf("%c", str[left]);
    prl (str, left + 1, right);
}
void prl (char *str, int left, int right) {
    if (left > right) return;
    printf("%c", str[right]);
    pm (str, left, right - 1);
}
void pm (char *str, int left, int right) {
    if (left > right) return;
    if ((left + right) % 2 == 0) {
        printf("%c", str[left]);
        plr (str, left + 1, right);
    } else {
        printf("%c", str[right]);
        prl (str, left, right - 1);
    }
}
```

Response

HDLROWOLLE

Exercise 11 (3.0 points)

Write the following C function:

```
void cde (int **mat, int r, int c);
```

That counts the number of distinct elements in a matrix. The function receives the matrix (*mat*) and its size (i.e., *r* rows and *c* columns) as parameters. It counts the number of distinct elements present in the matrix. For example, if the input matrix *mat* has four rows ($r = 4$) and three columns ($c = 3$) and stores the following values:

1	2	3
4	2	1
3	5	6
7	3	3

The function has to display (on standard output) something looking like the following:

```
1[2], 2[2], 3[4], 4[1], 5[1], 6[1], 7[1]
```

meaning that the value 1 appears 2 times, the value 2 appears 2 times, the value 3 appears 4 times, etc.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

```
// Only function cde had to be implemented.
#include <stdio.h>
#include <stdlib.h>
#define R 4
#define C 3
void cde(int[][C], int, int);
int main () {
    int mat[R][C] = {{1,2,3},{4,2,1},{3,5,6},{7,3,3}};
    cde(mat, R, C);
}
```

```

    return (1);
}
void cde(int mat[][C], int r, int c) {
    int i, ii, j, jj, count;
    int **flag;
    flag = malloc (r * sizeof (int *));
    if (flag == NULL) {
        fprintf (stderr, "Allocation error.\n");
        exit (0);
    }
    for (i=0; i<r; i++) {
        // NO need to INIT (calloc)
        flag[i] = calloc (c, sizeof (int));
        if (flag[i] == NULL) {
            fprintf (stderr, "Allocation error.\n");
            exit (0);
        }
    }
    // Traverse the matrix and update counts
    for (i=0; i<r; i++) {
        for (j=0; j<c; j++) {
            if (flag[i][j] == 0) {
                count = 0;
                for (ii=i; ii<r; ii++) {
                    for (jj=(ii==i)?j:0; jj<c; jj++) {
                        if (mat[ii][jj]==mat[i][j]) {
                            flag[ii][jj] = 1;
                            count ++;
                        }
                    }
                }
                fprintf (stdout, "%d[%d] ", mat[i][j], count);
            }
        }
        fprintf (stdout, "\n");
    }
    for (i=0; i<r; i++) {
        free (flag[i]);
    }
    free (flag);
    return;
}

```

Exercise 12 (5.0 points)

Write the following C function:

```
tree_t *fill (tree_t *root, char *s);
```

That receives the root pointer to a binary tree and a string. The binary tree includes nodes storing dynamic strings and (like a heap) maintains level-order completeness, i.e., all levels are fully filled from left to right. The function must insert the new string *s* on a new node linked as the leftmost non-existing element of the first tree level not yet completed. In other words, each new insertion must be placed on the last incomplete level, trying to complete it from left to right.

Define the type `tree_t` to implement the tree using a definition adopting `typedef`.

Suggestion: Notice that trees can be visited using a BFS strategy not just a DFS approach.

Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation

Solution

To be done.

Exercise 13 (9.0 points)

Given a 2D grid representing a sensor network in which each sensor covers a specific range of grid cells, we must find the **minimum** number of sensors that cover **all** cells (if possible). To do so, write the following C function:

```
int maximize_sensor_coverage (
    int **grid, int r, int c, int *sensors, int *count
);
```

The input parameters are the following: The matrix `grid` represents the areas covered by all sensors (i.e., each element is 1 if the corresponding cell is covered), and variables `r` and `c` are the size of this matrix. The output parameters are the following: The array `sensors` stores the selected sensors to cover all the cells, and `count` is the size of this array. The function returns 1 if it there exists a set of sensors covering all the cells, and zero otherwise.

For example, if `grid` represents five sensors (numbered from zero to four) covering (one) or not covering (zero) seven cells (numbered from zero to six):

cells	0	1	2	3	4	5	6
sensor 0	1	1	0	0	0	0	1
sensor 1	0	0	1	1	1	0	0
sensor 2	1	0	0	1	0	1	1
sensor 3	0	1	0	1	0	1	0
sensor 4	0	0	1	1	1	1	1

All cells (columns, from zero to six) can be covered with sensors (rows, from zero to four) 0, 1, and 2 but also with sensors 0 and 4. Thus, the function must set `sensors = {0, 4}`, `count = 2`, and return 1. Write the entire function using standard C libraries but implement all required personal libraries. Modularize the function adequately and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward code, complex or impossible to understand, will be penalized in terms of the final evaluation.

Solution

To be done.